

Error Handling

 systemdesignschool.io/fundamentals/pipeline-error-handling



Production data pipelines encounter failures. Network partitions. Schema mismatches between producers and consumers. Malformed records from buggy upstream systems. External services that rate-limit or timeout. These aren't exceptional—they're expected in production environments.

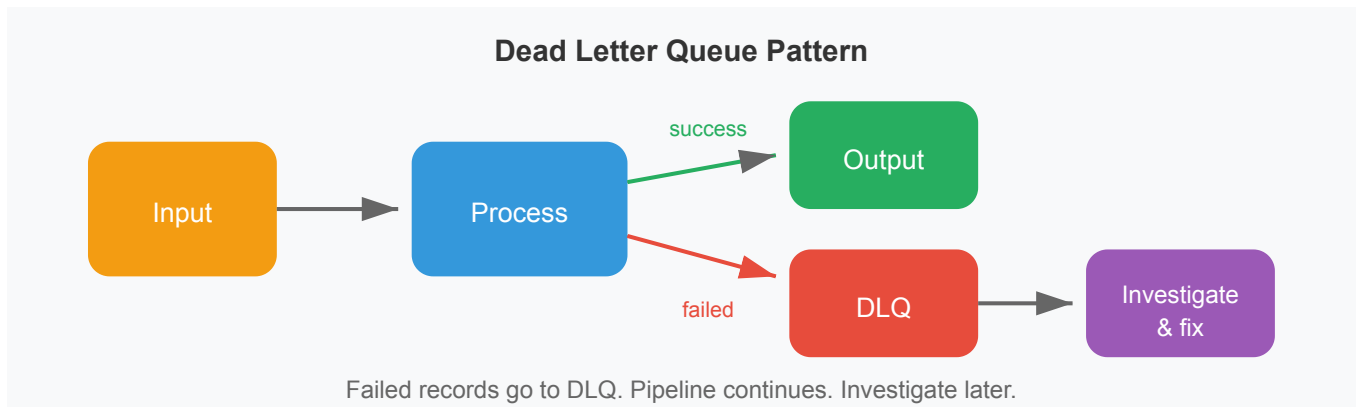
A robust pipeline handles failures gracefully: don't lose data, don't block progress on unrelated records, surface problems for investigation. This article covers the patterns that make this possible.

Dead Letter Queues: Don't Drop, Don't Block

When a record can't be processed, you have three choices:

1. **Drop it.** Simple but dangerous—you lose data silently.
2. **Block the pipeline.** Process nothing until the problem is fixed. One bad record stops everything.
3. **Sideline it.** Set the bad record aside and continue processing valid records.

The third option uses a **dead letter queue (DLQ)**—a separate storage location for records that failed processing.



The pipeline attempts to process each record. Successful records flow to the normal output. Failed records go to the DLQ with metadata about why they failed. The pipeline continues making progress on valid data.

Later, operators investigate the DLQ:

- Some failures are transient (timeout, rate limit)—replay these and they may succeed
- Some failures are data bugs (malformed JSON)—fix upstream and discard
- Some failures reveal pipeline bugs—fix the code and reprocess

Enriching DLQ entries: Include the original record, error message, timestamp, pipeline version, and which stage failed. This metadata speeds debugging. You might also categorize errors (validation failure, serialization error, downstream timeout) to help prioritize.

Common Failure Categories

Different failures need different handling:

Schema validation failures: A record has a null field where the schema expects a value. Or a string where it expects a number. These typically indicate producer bugs or schema drift—fix the source.

Business rule violations: A quantity is negative. A timestamp is in the future. A user ID doesn't exist. These may be valid data that your pipeline doesn't handle, or upstream bugs.

Transient downstream failures: The database times out. An API rate-limits you. A service is briefly unavailable. These often succeed on retry.

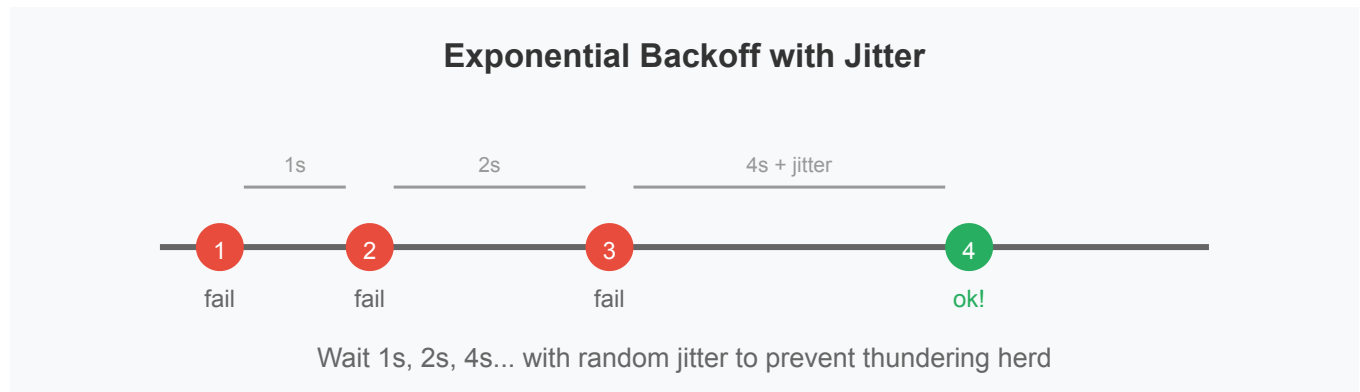
Serialization/parsing errors: Can't decode JSON. Can't deserialize Avro. Often indicates version mismatch between producer and consumer schemas.

Retry Strategies

Transient failures often succeed on retry. But how you retry matters.

Immediate retry (usually wrong): The downstream service timed out because it's overloaded. Retrying instantly adds more load. The service stays overloaded. You create a feedback loop of failure.

Exponential backoff: Space retries out. Wait 1 second, then 2, then 4, then 8. This gives the downstream service time to recover while still making progress.



Jitter: Add randomness to retry timing. Without jitter, if 100 workers all fail at the same time, they all retry at the same time (thundering herd). Each retry wave fails again. With jitter, retries spread out: worker 1 waits 1.3s, worker 2 waits 0.9s, worker 3 waits 1.7s.

```
retry_delay = base_delay * (2 ^ attempt) + random(0, base_delay)
```

Circuit breaker: If a downstream dependency is broken, stop calling it. After enough failures (e.g., 50% failure rate over 10 seconds), "open" the circuit—fail fast without actually calling the dependency. Periodically try again. If it succeeds, "close" the circuit and resume normal operation.

This protects both you (don't waste resources on calls that will fail) and the dependency (don't pile more load on an overloaded service).

Configuring Max Retries

Set a limit on retry attempts. Without a limit, records can retry forever, consuming resources and hiding the underlying problem.

A typical pattern:

1. Attempt processing
2. On failure, retry up to 3 times with exponential backoff and jitter
3. After exhausting retries, write to DLQ
4. Alert operators
5. Continue processing other records

The exact numbers depend on your use case. Short-lived transient failures (network blips) recover in 2-3 retries. Longer outages (service deployment, database failover) might need more retries or longer backoff.

Monitoring: Making Failures Visible

Pipelines run continuously without direct user interaction. Failures must surface through monitoring rather than user complaints.

Lag monitoring: How far behind real-time is the pipeline? If events happened at 10:00 but the pipeline is only processing events from 9:45, lag is 15 minutes. Growing lag means processing isn't keeping up—either throughput dropped or input volume increased.

Error rate: What fraction of records fail processing? A sudden spike indicates systemic problems: bad deployment, schema change, dependency outage. A slow increase might indicate data quality degradation upstream.

DLQ depth: How many records are waiting in the dead letter queue? A non-zero DLQ is normal. Rapid growth indicates an ongoing problem needing investigation.

Data quality checks: Output properties should match expectations. Record counts align with inputs (accounting for expected filtering). Values fall within valid ranges. Duplicate detection finds expected rates. These checks catch bugs that don't cause failures but produce wrong results.

End-to-end latency: Time from when an event was created to when it's available in the serving layer. This is what users experience. Even if each pipeline stage is fast, the sum might violate SLAs.

Treating Pipelines as Production Systems

Data pipelines deserve the same operational rigor as user-facing applications:

- **Dashboards** showing health metrics at a glance
- **Alerts** on degradation before users notice
- **On-call rotations** with clear escalation paths
- **Runbooks** for common failures
- **Capacity planning** to handle growth

A silent pipeline failure that corrupts analytics for a week is often worse than a user-facing outage that gets fixed in minutes.

What's Next

Error handling keeps pipelines running. But what about the serving layer—how do you make analytical data available with low latency? The next section covers **real-time analytics**: materialized views, time-series patterns, and streaming analytics architectures.